Claude Code for Accounting Research

Module 3 — EDGAR Text: Structured Data, and Your First Skill

John Barrios

Yale School of Management

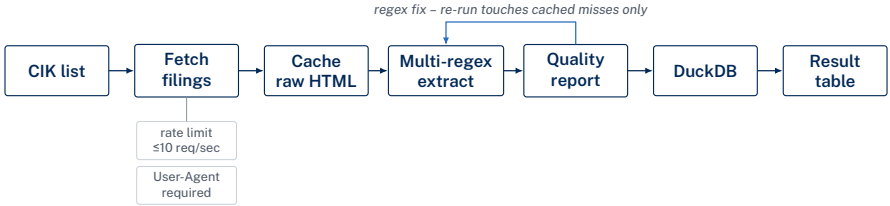
Today's Arc

- **0:00–0:03** Checkpoint follow-up: Duo triage results from Module 2
- **0:03–0:40** Lecture: EDGAR mechanics, text pipeline, measurement validity, skills
- **0:40–1:15** Live demo: two acts — EDGAR pipeline, then SKILL.md live-build
- **1:15–1:25** Break
- **1:25–2:45** Lab: pipeline $\rightarrow$ result table $\rightarrow$ your first skill
- **2:45–3:05** Debrief, exit ticket, bridge to Module 4

Two Halves, One Lesson

- Text as data: where AI meets accounting most naturally, and where it is most dangerous
- Half one: build the pipeline, CIK $\rightarrow$ fetch $\rightarrow$ extract $\rightarrow$ DuckDB $\rightarrow$ measure
- Half two: write the procedure you just worked out as a `SKILL.md`
- Both halves rest on Module 1's mantra: files persist, context doesn't

EDGAR Mechanics



- CIK: stable firm ID, assigned once, never reused
- `efts.sec.gov` full-text search + `data.sec.gov/submissions` per-filer endpoint
- Hard limits: 10 req/sec, mandatory `User-Agent` (name + email)

Vague vs. Front-Loaded

Vague:

```
"get me the risk factors from some 10-Ks"
```

Front-loaded:

```
"Fetch Item 1A from the 10-Ks filed by the CIKs in pinned_ciks.csv, FY2021--2023,
via EDGAR full-text search + submissions endpoints. 10 req/sec, User-Agent set.
Cache raw HTML to data/filings/."
```

Same task. The difference is whether facts you already know are in the prompt, or discovered by trial and error.

10-K Anatomy: Why One Regex Fails

- Item 1A and Item 7 nominally have one heading each; filers do not cooperate
- Four ways to write “Item 1A”: caps, trailing period, split TOC anchor, cross-reference decoy
- Fix: an ordered fallback chain, not a cleverer single pattern
- Log which pattern fired for each filing; silent failure is the real danger

Pipeline Robustness

- Cache raw HTML on first download; extraction then runs only on local files
- Fix a regex, re-run: zero new network requests if caching worked
- Quality report every pass: successes, failures, suspiciously-short flags
- Most useful prompt: *ask Claude why N filings are missing*

Common Failure

Treating the quality-report number as the deliverable

"99% extraction" means nothing without the denominator. If the hardest 1% of filings differ systematically from the ones that extracted cleanly, a pipeline that silently drops them has a selection problem.

DuckDB as the Research Store

- Single file, zero configuration, reads/writes from Python, R, or the CLI
- Three tables: `filings`, `sections`, `keywords`
- Plain English to SQL, but read the query before you trust the numbers
- Replaces a folder of forty CSVs with one file any coauthor can open

Keyword Counts $\neq$ Constructs

- "Tariff" mentioned 14 times, up from 6: rising exposure, or a boilerplate update?
- A keyword count measures word frequency; "exposure" is a construct about the firm
- The gap between the two is construct validity, a common way for a disclosure measure to be wrong and still survive peer review

Boilerplate and Year-over-Year Similarity

- Filers reuse risk-factor language year over year, far more than “disclosure” suggests
- Detectable signal: YoY textual similarity (Jaccard / cosine)
- High similarity + rising keyword count $\approx$ boilerplate inflation
- Low similarity + rising keyword count $\approx$ a real, rewritten disclosure
- These are diagnostics, not proof; they don’t replace the audit sample

The Human Audit Sample

- **Stratified, not drawn for convenience** — high/mid/low measure, size, industry; $\sim$50 filings
- **Rubric decided in advance** — written before reading, not adjusted filing by filing
- **Inter-rater agreement reported** — or state plainly that only one person coded it

Verify This

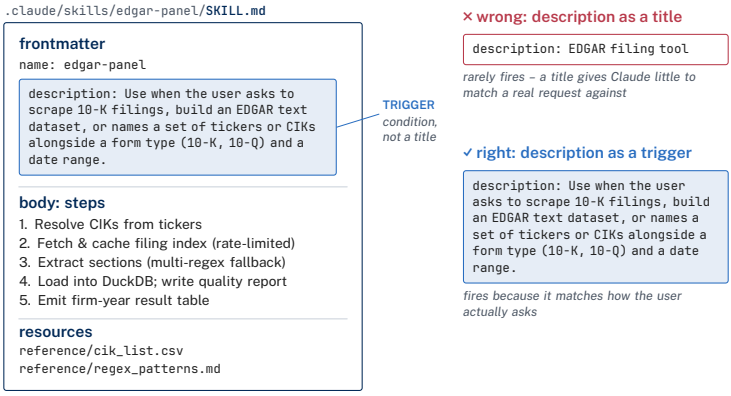
The protocol is the deliverable, not the measure

Fifty filings can't prove a measure valid. The lab asks for the protocol you would run before trusting it: which fifty, stratified how, coded against what rubric, by whom.

Skills Preview: Your SOP, Packaged

- Everything painful in the pipeline (front-loading, fallback chain, caching, validity check) is a standard operating procedure
- A skill turns something you re-explain every session into something a coauthor can run
- The pipeline's steps are already the outline of your skill

Anatomy of a SKILL.md



Only the description loads every session; the body loads when the skill runs. Skills respond to requests. They do not run on a schedule.

Description as Trigger, Not Title

Wrong (looks correct, rarely fires):

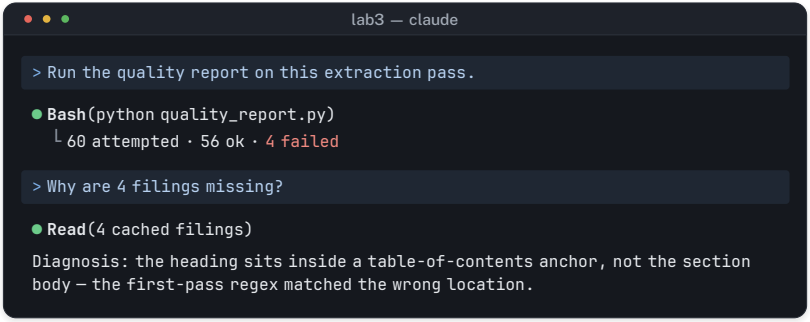
```
description: EDGAR filing tool
```

Right (names what a real request looks like):

```
description: Use when the user asks to scrape 10-K filings, build
an EDGAR text dataset, or names tickers/CIKs alongside a form type
and date range.
```

The procedure underneath is the same. A title describes the skill; a trigger gives the matching logic phrases it can recognize.

What You'll See: the Quality Report



Stylized session transcript (not a screenshot).

Illustrative numbers. Read what the quality report flagged before you re-run anything.

Successes, failures, and the suspiciously-short flags that turn a silent extraction gap into a diagnosable one.

Next: Demo Act 1 — EDGAR

0:40–1:00, compressed

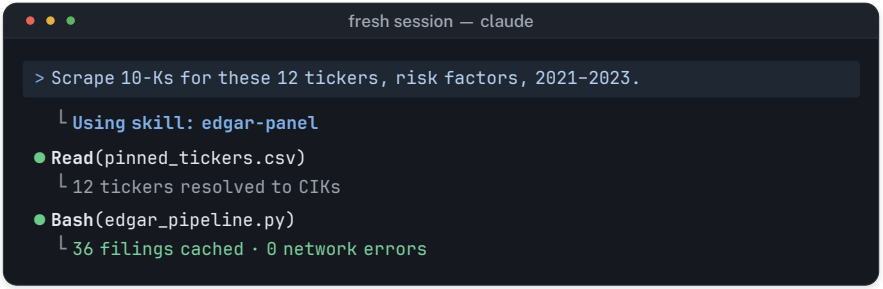
- Opening contrast: vague vs. front-loaded; count the rounds (5 min)
- Plan Mode in practice: full plan $\rightarrow$ `plan.md` $\rightarrow$ clear context $\rightarrow$ execute (4 min)
- Phases 1–2 live: CIK resolution + filing index, $\sim$8 firms (5 min)
- Phases 3–4, narrated: quality report, regex fix, the two banks, re-run: $\sim$85–90% $\rightarrow$ $\sim$99% (6 min)

Next: Demo Act 2 — SKILL.md

1:00–1:15, live-build

- Five-step workflow, run live: notice → name inputs → design outputs → draft `SKILL.md` → test-fire (10 min)
- Deliberate error: a description written as a title does not fire; rewritten as a trigger, it fires (5 min)

A Skill, Firing



Stylized session transcript (not a screenshot).

A fresh session, no re-explaining: the description field fires on the request's own words.

A fresh session recognizes the trigger-condition description and runs the packaged procedure without re-explanation.

A Second Skill: Reading a Paper

- Every paper you read produces the same notes, saved to a file: question, setting, sample and N, identifying variation, main result, replication
- Long PDFs: split into sections, read in order
- Every number cites a table and column; you check each one

Why the check matters

A wrong number reads as fluently as a right one: an N from an earlier filter, or a coefficient from the wrong column.

Splitting long PDFs: Cunningham's `split-pdf` skill (MixtapeTools). Identification-first template: Spina's skills brownbag.

Next: The Lab (1:25–2:45)

- **Part A (45 min):** pipeline — $\sim$20 firms $\times$ 3 years $\rightarrow$ DuckDB, one keyword topic
- **Part B (15 min):** validity — YoY similarity classification + 3-sentence audit protocol
- **Part C (20 min):** the skill — build `edgar-panel/SKILL.md`, test-fire fresh, install a catalog skill (stretch: `paper-reader`)

Lab Verify Checklist

- ☐ Cache works: a re-run downloads none of the already-fetched filings
- ☐ Quality report: extraction rate stated and $\geq$95%; every failure has a one-line diagnosis
- ☐ Result table reproduced by named scripts from README
- ☐ Validity paragraph + audit-sample protocol written
- ☐ `SKILL.md` description is a trigger, and it fires in a fresh session
- ☐ Rate limit respected, User-Agent set; show the code line

Debrief and Bridge to Module 4

- Extraction-rate leaderboard; genuine edge cases vs. fixable regex gaps
- Read 2–3 student skill descriptions aloud — would they trigger?
- Bridge: your text panel has CIKs; fundamentals live in Compustat, keyed by `gvkey`. Module 4 uses the crosswalk through an MCP server on your own WRDS account
- Exit ticket: standard 3 + “what would your second skill be?”